# Student Performance Prediction

Practical AI/ML Week 1 Project

4th-Year Computer Science Student
AI/ML Specialization

September 28, 2026

**Abstract**

This document presents an end-to-end Machine Learning project for Student Performance Prediction. The project demonstrates practical Python programming, data loading, data cleaning, exploratory data analysis, feature engineering, preprocessing, model training, model evaluation, prediction, visualization, and model serialization.

The implementation uses Python, Pandas, NumPy, Matplotlib, and Scikit-learn. Two regression models are trained and compared: Linear Regression and Random Forest Regression.

## Contents

**1 Project Overview** **3**

**2 Development Environment** **3**

**3 Dataset** **3**

**4 Target Variable** **4**

**5 Target Leakage Prevention** **4**

**6 Data Validation and Cleaning** **5**

**7 Data Preprocessing** **5**

**8 Training and Testing** **6**

**9 Machine Learning Models** **6**
9.1 Linear Regression . . . . . 6
9.2 Random Forest Regression . . . . . 7

**10 Model Evaluation** **7**
10.1 Mean Absolute Error . . . . . 7
10.2 Mean Squared Error . . . . . 7
10.3 Root Mean Squared Error . . . . . 7
10.4 R-squared . . . . . 8

**11 Model Selection** **8**

**12 Test Predictions** **8**

**13 Exploratory Data Analysis** **8**

**14 Visualizations** **9**

**15 Feature Importance** **9**

**16 Model Serialization** **10**

**17 Generated Outputs** **10**

**18 Professional Machine Learning Workflow** **10**

**19 Code Explanation by Major Section** **12**

**20 Project Limitations** **12**

**21 Possible Interview Questions** **12**

**22 Future Improvements** **13**

**23 Conclusion** **13**

# 1 Project Overview

The objective of this project is to build a practical Machine Learning solution that predicts student academic performance from the information available in the dataset.

The main project pipeline is:

**Data → Data Cleaning → EDA → Feature Engineering → Preprocessing → Model Training → Evaluation → Prediction**

The project demonstrates:

- Python programming
- Pandas data manipulation
- NumPy numerical operations
- Data visualization using Matplotlib
- Data cleaning
- Feature engineering
- Categorical feature encoding
- Machine Learning pipelines
- Regression modelling
- Model evaluation
- Model serialization

The report is intentionally condensed. The original project report contains a full Python source listing; this version retains the important implementation logic without reproducing every source-code line. :chatgpt-content-referenceindex="1"

# 2 Development Environment

The project uses the following technology stack:

| Technology | Purpose |
|---|---|
| Python | Main programming language |
| VS Code | Development environment |
| Jupyter Notebook | Data exploration and experimentation |
| GitHub | Version control and portfolio management |
| Pandas | Data manipulation |
| NumPy | Numerical operations |
| Matplotlib | Data visualization |
| Scikit-learn | Machine Learning |
| Joblib | Saving the trained model |

Table 1: Technology Stack

The implementation uses project-relative paths so that the dataset and generated outputs remain organized inside the project folder.

# 3 Dataset

The dataset is stored at:

`data/exams.csv`

The dataset contains eight columns:

| Column | Description |
|---|---|
| `gender` | Student gender |
| `race/ethnicity` | Student demographic group |
| `parental level of education` | Parent/guardian education level |
| `lunch` | Lunch category |
| `test preparation course` | Test preparation status |
| `math score` | Mathematics examination score |
| `reading score` | Reading examination score |
| `writing score` | Writing examination score |

Table 2: Dataset Features

# 4 Target Variable

The project creates a new target variable named `overall_score`.

It is calculated as:

$$\text{overall_score} = \frac{\text{math score} + \text{reading score} + \text{writing score}}{3}$$

For example, if:

$$\text{Math} = 60, \qquad \text{Reading} = 70, \qquad \text{Writing} = 80$$

then:

$$\text{overall_score} = \frac{60 + 70 + 80}{3} = 70$$

The project also derives a `performance_category` for interpretation:

| Score | Category |
|---|---|
| 70 and above | High |
| 50 to below 70 | Medium |
| Below 50 | Low |
| Missing | Unknown |

Table 3: Performance Categories

The performance category is for interpretation rather than the main machine-learning target. :chatgpt-content-referenceindex="2"

# 5 Target Leakage Prevention

Target leakage is an important Machine Learning consideration in this project.

The target is directly calculated from:

- math score
- reading score
- writing score

Therefore, these three variables are deliberately excluded from the model input features.
The model instead uses:

- gender
- race/ethnicity
- parental level of education
- lunch
- test preparation course

Using the three subject scores as model inputs would allow the model to see information that directly determines the target and would lead to an unrealistic evaluation. :chatgpt-content-referenceindex="3"

In Machine Learning notation:

$$X = [\text{gender}, \text{race/ethnicity}, \text{parental education}, \text{lunch}, \text{test preparation}]$$

$$y = \text{overall_score}$$

Here, $X$ represents the independent variables or features, while $y$ represents the dependent variable or target.

# 6 Data Validation and Cleaning

The implementation performs explicit data-quality checks before model training.

The workflow includes:

1. Check that the dataset exists.
2. Load the dataset.
3. Validate all expected columns.
4. Convert the three score columns to numeric values.
5. Check for missing values.
6. Check for duplicate rows.
7. Check score ranges.
8. Replace invalid scores outside the 0–100 range with NaN.
9. Remove rows where all three subject scores are missing.
10. Save the cleaned dataset.

The cleaned data is saved as:

`outputs/cleaned_student_data.csv`

These checks help prevent incorrect data types, invalid score values, and schema problems from silently affecting the model.

# 7 Data Preprocessing

The selected Machine Learning algorithms cannot directly process categorical strings.

Therefore, the project uses:

1. SimpleImputer for missing categorical values.
2. OneHotEncoder for categorical encoding.
3. ColumnTransformer to apply preprocessing to selected columns.

4. Pipeline to combine preprocessing and Machine Learning.

The encoder is configured using:

```
OneHotEncoder(handle_unknown="ignore")
```

The setting `handle_unknown="ignore"` ensures that an unseen categorical value during prediction does not cause the transformation to fail.

For example:

```
gender = female
```

is converted into numerical indicator variables.

Using a Pipeline also ensures that the same transformation logic is used during training and prediction. :chatgpt-content-referenceindex="4"

# 8 Training and Testing

The dataset is divided into:

- 80% training data
- 20% testing data

The split uses:

```
test_size=0.20
random_state=42
```

The fixed random state makes the split reproducible.

The training set is used to learn the model parameters, while the testing set is reserved for evaluating the model on unseen examples.

The explicit feature configuration is:

```
feature_columns = [
    "gender",
    "race/ethnicity",
    "parental level of education",
    "lunch",
    "test preparation course",
]

target_column = "overall_score"
```

# 9 Machine Learning Models

Two regression models are implemented.

## 9.1 Linear Regression

Linear Regression is used as the baseline model.

Its conceptual form is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where:

- $\hat{y}$ is the predicted score.
- $\beta_0$ is the intercept.
- $\beta_i$ are learned coefficients.
- $x_i$ are the input features.

The baseline provides a simple reference point for model comparison.

### 9.2 Random Forest Regression

The second model is a Random Forest Regressor.

A Random Forest is an ensemble of decision trees.

The implementation uses:

```
RandomForestRegressor(
    n_estimators=300,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
```

The main parameters mean:

- `n_estimators=300`: 300 decision trees.
- `min_samples_split=5`: minimum samples required to split an internal node.
- `min_samples_leaf=2`: minimum samples allowed in a leaf.
- `n_jobs=-1`: use available CPU cores.

## 10 Model Evaluation

The project evaluates both models using:

### 10.1 Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAE represents the average absolute prediction error.

Lower values indicate smaller average errors.

### 10.2 Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE penalizes larger errors more strongly because the errors are squared.

### 10.3 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE is expressed in the same units as the target variable.

### 10.4 R-squared

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R-squared measures how well the model explains variation in the target variable.

The code evaluates the models using all three metrics. :chatgpt-content-referenceindex="5"

## 11 Model Selection

The models are stored in a comparison DataFrame.

The selection rule is:

**Choose the model with the lowest RMSE.**

The comparison is saved as:

`outputs/model_comparison.csv`

The selected pipeline is then used to make predictions on the held-out test data.

The source explicitly selects the model by finding the minimum RMSE. :chatgpt-content-referenceindex="6"

## 12 Test Predictions

Predictions are generated for the test set.

The saved prediction table contains:

- Actual Score
- Predicted Score
- Absolute Error

The resulting file is:

`outputs/test_predictions.csv`

This provides a record of individual model predictions and prediction errors.

## 13 Exploratory Data Analysis

The project performs several exploratory analyses.

These include:

- Dataset dimensions.
- Data types.
- Missing values.
- Duplicate rows.
- Score ranges.
- Summary statistics.
- Categorical distributions.
- Average scores.
- Grouped performance analysis.

The grouped analyses include:

- Overall score by test preparation course.
- Overall score by gender.
- Overall score by parental education.
- Overall score by lunch type.

For grouped analyses, the implementation calculates values such as count, mean, and median. :chatgpt-content-referenceindex="7"

## 14 Visualizations

The project creates the following visualizations:

1. Overall score distribution.
2. Average score by subject.
3. Average overall score by test preparation.
4. Average overall score by gender.
5. Average overall score by parental education.
6. Average overall score by lunch type.
7. Actual versus predicted scores.

The graphs are saved in the `outputs` directory.
Important output files include:

| File | Purpose |
|---|---|
| `overall_score_distribution.png` | Overall score distribution |
| `average_subject_scores.png` | Subject average scores |
| `score_by_test_preparation.png` | Preparation analysis |
| `score_by_gender.png` | Gender analysis |
| `score_by_parental_education.png` | Education analysis |
| `score_by_lunch.png` | Lunch analysis |
| `actual_vs_predicted.png` | Prediction comparison |

Table 4: Generated Visualization Files

## 15 Feature Importance

The Random Forest model provides feature importance information.

Because the categorical variables are one-hot encoded, the resulting feature names represent transformed categorical columns.

Feature importance can help explain which transformed inputs were useful for prediction.

However:

**Feature importance does not automatically imply causation.**

A highly important feature means that it helped the trained model make predictions. It does not by itself prove that the feature caused the student's result.

The project saves the feature importance information as:

`outputs/feature_importance.csv`

## 16 Model Serialization

The best trained model pipeline is saved using Joblib.

The output file is:

`outputs/student_performance_model.pkl`

This allows the trained model to be reused later without retraining.

A saved model can conceptually be loaded with:

```
model = joblib.load("student_performance_model.pkl")
```

Predictions can then be generated using:

```
model.predict(new_data)
```

The original report documents the model serialization and reuse process. :chatgpt-content-referenceindex="8"

## 17 Generated Outputs

The project generates the following files:

| File | Purpose |
|---|---|
| `cleaned_student_data.csv` | Cleaned dataset |
| `model_comparison.csv` | Model comparison |
| `test_predictions.csv` | Actual and predicted values |
| `feature_importance.csv` | Random Forest feature importance |
| `student_performance_model.pkl` | Serialized model |
| `project_summary.txt` | Project summary |
| `overall_score_distribution.png` | Score histogram |
| `average_subject_scores.png` | Average subject scores |
| `score_by_test_preparation.png` | Test preparation analysis |
| `score_by_gender.png` | Gender analysis |
| `score_by_parental_education.png` | Education analysis |
| `score_by_lunch.png` | Lunch analysis |
| `actual_vs_predicted.png` | Prediction comparison |

Table 5: Project Outputs

## 18 Professional Machine Learning Workflow

The complete workflow is:

`exams.csv`

↓

`Data Validation`

↓

Data Cleaning

↓

Feature Engineering

↓

Exploratory Data Analysis

↓

Feature Selection

↓

Train/Test Split

↓

Preprocessing

↓

Linear Regression + Random Forest

↓

Model Evaluation

↓

Model Comparison

↓

Best Model

↓

Prediction

↓

Save Model and Results

# 19 Code Explanation by Major Section

| Section | Purpose |
|---|---|
| Imports | Loads pathlib, NumPy, Pandas, Matplotlib, Scikit-learn, and Joblib. |
| Paths | Creates project-relative paths for the dataset and outputs. |
| Dataset Loading | Uses Pandas to load `exams.csv`. |
| Validation | Checks whether all required columns exist. |
| Cleaning | Checks missing values, duplicates, ranges, and invalid scores. |
| Feature Engineering | Creates `overall_score` and `performance_category`. |
| EDA | Calculates distributions, statistics, and grouped comparisons. |
| Preprocessing | Uses imputation and one-hot encoding in a Pipeline. |
| Training | Trains Linear Regression and Random Forest Regression. |
| Evaluation | Calculates MAE, RMSE, and R-squared. |
| Model Selection | Chooses the model with the smallest RMSE. |
| Prediction | Predicts unseen test examples and an example student profile. |
| Serialization | Saves the trained model with Joblib. |

Table 6: Major Code Sections

# 20 Project Limitations

The dataset does not contain:

- Study hours.
- Attendance.
- Previous grades.
- Longitudinal academic history.

Therefore, the project cannot evaluate the predictive value of those factors.

The project should be treated as a practical Machine Learning learning project rather than a production-ready student-risk system.

Another important limitation is that the target is constructed from the three exam scores themselves. Therefore, the model estimates the overall exam result from student-profile variables instead of predicting a future exam before those scores exist. :chatgpt-content-referenceindex="9"

# 21 Possible Interview Questions

1. Why is this problem formulated as regression?
2. What is the target variable?
3. What are the input features?
4. Why are math, reading, and writing excluded from the model?
5. What is target leakage?
6. Why do categorical features need encoding?
7. What does OneHotEncoder do?
8. Why is Pipeline useful?
9. Why split data into training and testing sets?
10. What is the purpose of random_state=42?
11. Why is Linear Regression used as a baseline?
12. Why was Random Forest tested?
13. What does MAE measure?
14. What does RMSE measure?

15. What does R-squared measure?
16. Why can RMSE be useful for score prediction?
17. What does n_estimators=300 mean?
18. What does n_jobs=-1 mean?
19. What is feature importance?
20. Does feature importance imply causation?
21. What limitations does the dataset have?
22. What additional features would make the model more useful?

## 22 Future Improvements

Potential improvements include:

- Add attendance information.
- Add study-hour information.
- Add previous grades.
- Add assignment performance.
- Add longitudinal student history.
- Test additional regression algorithms.
- Use cross-validation.
- Perform hyperparameter tuning.
- Build a Streamlit user interface.
- Add automated model monitoring.
- Deploy the application.

## 23 Conclusion

This project demonstrates a complete introductory Machine Learning workflow rather than only training a model.

The implementation starts with raw data and performs:

**Validation → Cleaning → Exploration → Feature Engineering → Preprocessing → Training → Evaluation → Prediction**

The project also demonstrates an important AI engineering principle: the quality of a Machine Learning solution depends not only on the algorithm but also on correct problem formulation, data preparation, leakage prevention, evaluation, reproducibility, and understanding of limitations.